

AI Scene Narrator for Accessibility

Technical Manual: Spatial Reasoning Engine (app/spatial.py) - From Scratch

1. Executive Overview & Problem Context

Traditional object detection models (e.g., YOLO) output isolated object classifications with bounding boxes (e.g., 'chair', 'table'). For visually impaired users, discrete label lists do not convey situational awareness. The user immediately needs to know: **Where is the object? What is it resting on? What is beside it? Is it in front of me?**

The **Spatial Reasoning Engine** acts as the critical bridge between raw detections and Natural Language Generation (NLG). It evaluates bounding-box geometry using deterministic 2D heuristics without requiring heavy 3D neural nets, LiDAR, or cloud connectivity. This ensures 100% offline edge execution on commodity CPU hardware with sub-millisecond latency.

2. System Architecture & Information Pipeline

The complete Scene Narrator architecture operates as a strict four-stage linear pipeline:

Stage 1: Detection Layer app/detection.py	YOLOv8n extracts labels, confidence scores, and bounding boxes (x1, y1, x2, y2) per frame.
Stage 2: Spatial Reasoning app/spatial.py	(Current Module) Analyzes bounding box intervals, centroids, ground contact base lines, and scale ratios to infer grounded predicates.
Stage 3: Natural Language Generation app/nlg.py	Transforms spatial predicates and frame zones into grammatically diverse spoken English sentences using templates and synonym banks.
Stage 4: Text-To-Speech (TTS) app/tts_engine.py	Synthesizes speech locally using pyttsx3 (SAPI5 on Windows / NSSpeechSynthesizer on macOS) for low-latency voice narration.

3. Geometric Heuristics & Predicate Definitions (from Scratch)

Given camera coordinates where (0, 0) is top-left, x increases rightward, and y increases downward, each detected bounding box A is represented by top-left (x1, y1) and bottom-right (x2, y2):

- **Centroids:** $cx = (x1 + x2) / 2$, $cy = (y1 + y2) / 2$
- **Dimensions:** $w = x2 - x1$, $h = y2 - y1$, $Area = w \times h$
- **1D Horizontal Overlap:** $overlap_x = \max(0, \min(A.x2, B.x2) - \max(A.x1, B.x1))$
- **1D Vertical Overlap:** $overlap_y = \max(0, \min(A.y2, B.y2) - \max(A.y1, B.y1))$
- **Horizontal Gap:** $gap_x = \max(0, \max(A.x1, B.x1) - \min(A.x2, B.x2))$

Detailed Predicate Rules:

Predicate	Geometric Conditions & Rationale	Assistive Example
on (Support)	1. Subject is above object ($A.cy < B.cy$) 2. High horizontal overlap: $overlap_x / A.w \geq 0.45$ 3. Base contact: $A.y2$ aligns near $B.y1$ within top margin $[-0.20 A.h, +0.35 B.h]$ 4. Plausibility: $A.Area \leq 1.8 \times B.Area$ (a sofa is not on a cup)	'A cup is on the desk.'
in front of (Depth Proxy)	1. In perspective, closer objects tend to have a lower base in frame ($A.y2 > B.y2$ by $\geq 0.06 \times \max(h)$) 2. Shared line of sight: $overlap_x / \min(w) \geq 0.15$ 3. Scale factor: Area ratio $\geq 1.6x$ or base difference $\geq 15\%$	'A person is in front of the couch.'
next to (Proximity)	1. Vertical plane alignment: vertical overlap ≥ 0.30 or $ A.cy - B.cy \leq 0.50 \times \max(h)$ 2. Horizontal proximity: $gap_x \leq 1.25 \times \min(A.w, B.w)$ 3. Not vertically stacked or resting on top	'A chair is next to the table.'
inside (Containment)	Subject bounding box is almost entirely contained inside object: $Intersection / A.Area \geq 0.82$ with $A.Area < 0.85 \times B.Area$.	'An apple is inside the bowl.'
to left / right of (Fallback)	When objects are horizontally distant without contact: $A.cx < B.cx$ evaluates to 'to the left of', suppressing pairs separated by $>75\%$ of frame width.	'A lamp is to the left of the bed.'
frame_zone() (Single Object)	Normalized centroid ($cx/W, cy/H$) mapped across a 3x3 matrix: Center, Left, Right, Top, Bottom, Top-Left, Top-Right, Bottom-Left, Bottom-Right.	'There is a laptop in the center.'

4. Anti-Clutter & Scene Graph Optimization

A scene with N objects generates N(N-1) ordered pair comparisons. Without filtering, an 8-object scene creates 56 relations, producing cognitive overload for a blind user. `app/spatial.py` employs three algorithmic optimizations:

- 1. **Salience Hierarchy Ranking:** Relationships are weighted by informational urgency: Support (1.0) > Containment (0.95) > Depth (0.85) > Proximity (0.80) > Directional (0.50).
- 2. **Reciprocal Deduplication:** Prunes inverse pairs so only the canonical primary relation is retained (e.g., outputs 'A is on B' rather than redundantly following up with 'B is below A').
- 3. **Transitive Support Suppression:** If object A is resting *on* object B, and B is *next to* C, any synthetic relation between A and C (e.g. 'cup next to chair') is suppressed because A is bound to B. This ensures narrations sound natural and human.

5. Step-by-Step Usage Guide (How to Use It Now)

Run everything from the project root (`ai-scene-narrator/`). Here is how to run and test it:

```
# 1. Run the full test suite (spatial, NLG, pipeline, API):
python -m pytest

# 2. Run the end-to-end demo (spatial + NLG on synthetic scenes):
python -m app.demo

# 3. Run the YOLO detector on a live webcam (press 'q' to quit):
python -m app.detection

# 4. Run the YOLO detector on an image file:
python -m app.detection --image path/to/image.jpg

# 5. Start the API (docs at http://localhost:8000/docs):
uvicorn app.main:app --port 8000
```

Calling the Spatial Module from Code:

```
from app.detection import Detection
from app.spatial import analyze_scene_spatial, SpatialConfig

# Create detections (or receive them from app.detection.detect(frame))
desk = Detection(label='desk', confidence=0.95, x1=100, y1=200, x2=550, y2=450)
laptop = Detection(label='laptop', confidence=0.92, x1=200, y1=160, x2=380, y2=220)

# Run spatial scene analysis
result = analyze_scene_spatial([desk, laptop], frame_width=640, frame_height=480)

print(result['structured_narration'])
# Output: 'A laptop is on the desk.'
print(result['primary_relations'])
# Output: [{'subject': 'laptop', 'predicate': 'on', 'object': 'desk', 'confidence': 0.91, ...}]
```

6. Engineering Hardening (v2.0)

- **Class-aware overlap preservation:** different object classes are never removed merely because their boxes overlap.
- **Conservative duplicate suppression:** same-label boxes are merged only at high IoU or near-total containment.
- **Ambiguity rejection:** near-vertical boxes are not forced into a left/right relation.
- **Evidence-bounded confidence:** relation confidence cannot exceed the weakest detector confidence.
- **Upload safety:** image dimensions and total pixels are checked before OpenCV decoding.
- **Operational readiness:** CPU work is threadpooled, `ready` exposes model readiness, and live signatures include movement.

7. Project Roadmap & Remaining Steps

All stages are implemented with a hardened production-oriented baseline:

- **app/nlg.py (Stage 3):** Template-based Natural Language Generation with synonym diversification.
- **app/tts_engine.py (Stage 4):** Offline pyttsx3 speech on a worker thread; degrades gracefully on headless servers.
- **app/pipeline.py:** *NarrationPipeline* links detection -> spatial -> NLG, plus a scene-change gate for live narration.

- **app/main.py**: FastAPI service (*/api/analyze, /api/narrate, /health*).
- **frontend/streamlit_app.py**: Streamlit UI (upload / camera snapshot, browser speech).
- **Dockerfile + docker-compose.yml**: one-command deployment of API and frontend.